

```

    }
    du=funcd.df(u);
    if (fu <= fx) {
        if (u >= x) a=x; else b=x;
        mov3(v,fv,dv,w,fw,dw);
        mov3(w,fw,dw,x,fx,dx);
        mov3(x,fx,dx,u,fu,du);
    } else {
        if (u < x) a=u; else b=u;
        if (fu <= fw || w == x) {
            mov3(v,fv,dv,w,fw,dw);
            mov3(w,fw,dw,u,fu,du);
        } else if (fu < fv || v == x || v == w) {
            mov3(v,fv,dv,u,fu,du);
        }
    }
}
throw("Too many iterations in routine dbrent");
};

```

Now all the housekeeping, sigh.

CITED REFERENCES AND FURTHER READING:

- Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 55; 454–458.[1]
- Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover), p. 78.

10.5 Downhill Simplex Method in Multidimensions

With this section we begin consideration of multidimensional minimization, that is, finding the minimum of a function of more than one independent variable. This section stands apart from those that follow, however: All of the algorithms after this section will make explicit use of a one-dimensional minimization algorithm as a part of their computational strategy. This section implements an entirely self-contained strategy, in which one-dimensional minimization does not figure.

The *downhill simplex method* is due to Nelder and Mead [1]. The method requires only function evaluations, not derivatives. It is not very efficient in terms of the number of function evaluations that it requires. Powell's method (§10.7) or the DFP method with finite differences (§10.9) is almost surely faster in all likely applications. However, the downhill simplex method may frequently be the *best* method to use if the figure of merit is “get something working quickly” for a problem whose computational burden is small.

The method has a geometrical naturalness about it that makes it delightful to describe or work through:

A *simplex* is the geometrical figure consisting, in N dimensions, of $N + 1$ points (or vertices) and all their interconnecting line segments, polygonal faces, etc. In two dimensions, a simplex is a triangle. In three dimensions, it is a tetrahedron, not necessarily the regular tetrahedron. (The *simplex method* of linear programming, described in §10.10, also makes use of the geometrical concept of a simplex. Otherwise

it is completely unrelated to the algorithm that we are describing in this section.) In general we are only interested in simplexes that are nondegenerate, i.e., that enclose a finite inner N -dimensional volume. If any point of a nondegenerate simplex is taken as the origin, then the N other points define vector directions that span the N -dimensional vector space.

In one-dimensional minimization, it was possible to bracket a minimum, so that the success of a subsequent isolation was guaranteed. Alas! There is no analogous procedure in multidimensional space. For multidimensional minimization, the best we can do is give our algorithm a starting guess, that is, an N -vector of independent variables as the first point to try. The algorithm is then supposed to make its own way downhill through the unimaginable complexity of an N -dimensional topography, until it encounters a (local, at least) minimum.

The downhill simplex method must be started not just with a single point, but with $N + 1$ points, defining an initial simplex. If you think of one of these points (it matters not which) as being your initial starting point $\mathbf{P}_0$, then you can take the other N points to be

$$\mathbf{P}_i = \mathbf{P}_0 + \Delta \mathbf{e}_i \quad (10.5.1)$$

where the $\mathbf{e}_i$'s are N unit vectors, and where Δ is a constant that is your guess of the problem's characteristic length scale. (Or, you could have different Δ_i 's for each vector direction.)

The downhill simplex method now takes a series of steps, most steps just moving the point of the simplex where the function is largest ("highest point") through the opposite face of the simplex to a lower point. These steps are called reflections, and they are constructed to conserve the volume of the simplex (and hence maintain its nondegeneracy). When it can do so, the method expands the simplex in one or another direction to take larger steps. When it reaches a "valley floor," the method contracts itself in the transverse direction and tries to ooze down the valley. If there is a situation where the simplex is trying to "pass through the eye of a needle," it contracts itself in all directions, pulling itself in around its lowest (best) point. The routine name *amoeba* is intended to be descriptive of this kind of behavior; the basic moves are summarized in Figure 10.5.1.

Termination criteria can be delicate in any multidimensional minimization routine. Without bracketing, and with more than one independent variable, we no longer have the option of requiring a certain tolerance for a single independent variable. We typically can identify one "cycle" or "step" of our multidimensional algorithm. It is then possible to terminate when the vector distance moved in that step is fractionally smaller in magnitude than some tolerance *tol*. Alternatively, we could require that the decrease in the function value in the terminating step be fractionally smaller than some tolerance *ftol*. Note that while *tol* should not usually be smaller than the square root of the machine precision, it is perfectly appropriate to let *ftol* be of order the machine precision (or perhaps slightly larger so as not to be confused by roundoff).

Note well that either of the above criteria might be fooled by a single anomalous step that, for one reason or another, failed to get anywhere. Therefore, it is frequently a good idea to *restart* a multidimensional minimization routine at a point where it claims to have found a minimum. For this restart, you should reinitialize any ancillary input quantities. In the downhill simplex method, for example, you should reinitialize N of the $N + 1$ vertices of the simplex again by equation (10.5.1), with

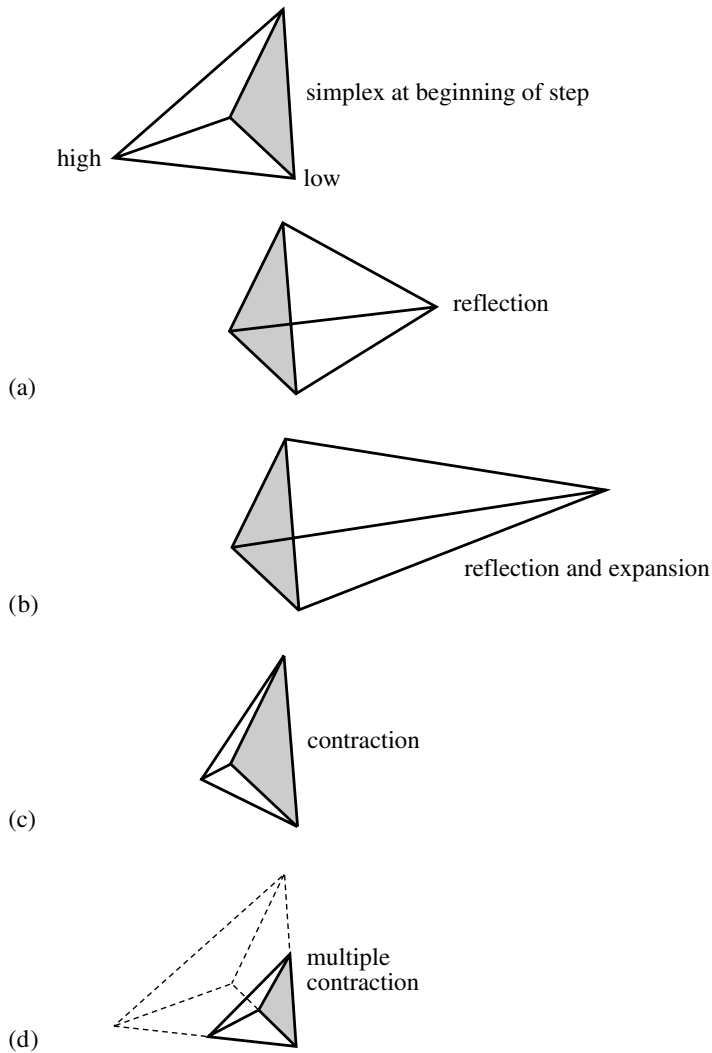


Figure 10.5.1. Possible outcomes for a step in the downhill simplex method. The simplex at the beginning of the step, here a tetrahedron, is shown, top. The simplex at the end of the step can be any one of (a) a reflection away from the high point, (b) a reflection and expansion away from the high point, (c) a contraction along one dimension from the high point, or (d) a contraction along all dimensions toward the low point. An appropriate sequence of such steps will always converge to a minimum of the function.

P_0 being one of the vertices of the claimed minimum.

Restarts should never be very expensive; your algorithm did, after all, converge to the restart point once, and now you are starting the algorithm already there.

The routine below has three different user interfaces. The simplest requires you to supply the initial simplex as in equation (10.5.1):

```
Amoeba am(ftol);
VecDoub point = ...; Doub del = ...;
pmin=am.minimize(point,del,func);
```

The value of the function at the minimum is available in `am.fmin`.

Second, you can use equation (10.5.1) with a vector of increments Δ_i :

```
VecDoub dels = ...;
pmin=am.minimize(point,dels,func);
```

Lastly, you can provide the simplex as an $(N + 1) \times N$ matrix whose rows are the coordinates of each vertex:

```
MatDoub p = ...;
pmin=am.minimize(p,func);
```

Consider, then, our N -dimensional amoeba:

```
struct Amoeba {
```

[amoeba.h](#)

Multidimensional minimization by the downhill simplex method of Nelder and Mead.

```
const Doub ftol;
Int nfunc;           The number of function evaluations.
Int mpts;
Int ndim;
Doub fmin;           Function value at the minimum.
VecDoub y;           Function values at the vertices of the simplex.
MatDoub p;           Current simplex.
```

```
Amoeba(const Doub ftol) : ftol(ftol) {}
```

The constructor argument `ftol` is the fractional convergence tolerance to be achieved in the function value (n.b.!).

```
template <class T>
```

```
VecDoub minimize(VecDoub_I &point, const Doub del, T &func)
```

Multidimensional minimization of the function or functor `func(x)`, where `x[0..ndim-1]` is a vector in `ndim` dimensions, by the downhill simplex method of Nelder and Mead. The initial simplex is specified as in equation (10.5.1) by a `point[0..ndim-1]` and a constant displacement `del` along each coordinate direction. Returned is the location of the minimum.

```
{
    VecDoub dels(point.size(),del);
    return minimize(point,dels,func);
}
```

```
template <class T>
```

```
VecDoub minimize(VecDoub_I &point, VecDoub_I &dels, T &func)
```

Alternative interface that takes different displacements `dels[0..ndim-1]` in different directions for the initial simplex.

```
{
    Int ndim=point.size();
    MatDoub pp(ndim+1,ndim);
    for (Int i=0;i<ndim+1;i++) {
        for (Int j=0;j<ndim;j++)
            pp[i][j]=point[j];
        if (i !=0 ) pp[i][i-1] += dels[i-1];
    }
    return minimize(pp,func);
}
```

```
template <class T>
```

```
VecDoub minimize(MatDoub_I &pp, T &func)
```

Most general interface: initial simplex specified by the matrix `pp[0..ndim][0..ndim-1]`. Its `ndim+1` rows are `ndim`-dimensional vectors that are the vertices of the starting simplex.

```
{
    const Int NMAX=5000;           Maximum allowed number of function evalua-
    const Doub TINY=1.0e-10;       tions.
    Int ihi,ilo,inhi;
    mpts=pp.nrows();
    ndim=pp.ncols();
    VecDoub psum(ndim),pmin(ndim),x(ndim);
    p=pp;
    y.resize(mpts);
    for (Int i=0;i<mpts;i++) {
        for (Int j=0;j<ndim;j++)
            x[j]=p[i][j];
        y[i]=func(x);
    }
```

```

}
nfunc=0;
get_psum(p,psum);
for (;;) {
    ilo=0;
    First we must determine which point is the highest (worst), next-highest, and
    lowest (best), by looping over the points in the simplex.
    ihi = y[0]>y[1] ? (inhi=1,0) : (inhi=0,1);
    for (Int i=0;i<mpts;i++) {
        if (y[i] <= y[ilo]) ilo=i;
        if (y[i] > y[ihi]) {
            inhi=ihi;
            ihi=i;
        } else if (y[i] > y[inhi] && i != ihi) inhi=i;
    }
    Doub rtol=2.0*abs(y[ihi]-y[ilo])/(abs(y[ihi])+abs(y[ilo])+TINY);
    Compute the fractional range from highest to lowest and return if satisfactory.
    if (rtol < ftol) {
        If returning, put best point and value in slot 0.
        SWAP(y[0],y[ilo]);
        for (Int i=0;i<ndim;i++) {
            SWAP(p[0][i],p[ilo][i]);
            pmin[i]=p[0][i];
        }
        fmin=y[0];
        return pmin;
    }
    if (nfunc >= NMAX) throw("NMAX exceeded");
    nfunc += 2;
    Begin a new iteration. First extrapolate by a factor -1 through the face of the
    simplex across from the high point, i.e., reflect the simplex from the high point.
    Doub ytry=amotry(p,y,psum,ihi,-1.0,func);
    if (ytry <= y[ilo])
        Gives a result better than the best point, so try an additional extrapolation
        by a factor 2.
        ytry=amotry(p,y,psum,ihi,2.0,func);
    else if (ytry >= y[inhi]) {
        The reflected point is worse than the second-highest, so look for an interme-
        diate lower point, i.e., do a one-dimensional contraction.
        Doub ysave=y[ihi];
        ytry=amotry(p,y,psum,ihi,0.5,func);
        if (ytry >= ysave) {
            Can't seem to get rid of that high point.
            for (Int i=0;i<mpts;i++) {
                Better contract around the lowest
                if (i != ilo) {
                    (best) point.
                    for (Int j=0;j<ndim;j++)
                        p[i][j]=psum[j]=0.5*(p[i][j]+p[ilo][j]);
                    y[i]=func(psum);
                }
            }
            nfunc += ndim;
            Keep track of function evaluations.
            get_psum(p,psum);
            Recompute psum.
        }
        } else --nfunc;
        Correct the evaluation count.
    }
    Go back for the test of doneness and the next
    iteration.
}
inline void get_psum(MatDoub_I &p, VecDoub_0 &psum)
Utility function.
{
    for (Int j=0;j<ndim;j++) {
        Doub sum=0.0;
        for (Int i=0;i<mpts;i++)
            sum += p[i][j];
        psum[j]=sum;
    }
}

```

```

template <class T>
Doub amotry(MatDoub_IO &p, VecDoub_O &y, VecDoub_IO &psum,
    const Int ihi, const Doub fac, T &func)
Helper function: Extrapolates by a factor fac through the face of the simplex across from
the high point, tries it, and replaces the high point if the new point is better.
{
    VecDoub ptry(ndim);
    Doub fac1=(1.0-fac)/ndim;
    Doub fac2=fac1-fac;
    for (Int j=0;j<ndim;j++)
        ptry[j]=psum[j]*fac1-p[ihi][j]*fac2;
    Doub ytry=func(ptry);           Evaluate the function at the trial point.
    if (ytry < y[ihi]) {           If it's better than the highest, then replace the
        y[ihi]=ytry;               highest.
        for (Int j=0;j<ndim;j++) {
            psum[j] += ptry[j]-p[ihi][j];
            p[ihi][j]=ptry[j];
        }
    }
    return ytry;
}
};

```

CITED REFERENCES AND FURTHER READING:

- Nelder, J.A., and Mead, R. 1965, "A Simplex Method for Function Minimization," *Computer Journal*, vol. 7, pp. 308–313.[1]
- Yarbro, L.A., and Deming, S.N. 1974, "Selection and Preprocessing of Factors for Simplex Optimization," *Analytica Chimica Acta*, vol. 73, pp. 391–398.
- Jacoby, S.L.S, Kowalik, J.S., and Pizzo, J.T. 1972, *Iterative Methods for Nonlinear Optimization Problems* (Englewood Cliffs, NJ: Prentice-Hall).

10.6 Line Methods in Multidimensions

We know (§10.2 – §10.4) how to minimize a function of one variable. If we start at a point $\mathbf{P}$ in N -dimensional space, and proceed from there in some vector direction $\mathbf{n}$, then any function of N variables $f(\mathbf{P})$ can be minimized along the line $\mathbf{n}$ by our one-dimensional methods. One can dream up various multidimensional minimization methods that consist of sequences of such line minimizations. Different methods will differ only by how, at each stage, they choose the next direction $\mathbf{n}$ to try. The minimization methods in the next two sections fall under this general schema of successive line minimizations. (The quasi-Newton algorithm in §10.9 does not need very accurate line minimizations. Accordingly, it uses the approximate line minimization routine, `lnsrch` from §9.7.1.)

In this section we provide the line minimization routine `linmin` as part of the base class `Linemethod` from which we will derive the minimization methods in the next two sections. These minimization routines regard `linmin` as a black-box subalgorithm, whose definition is

`linmin`: Given as input the vectors $\mathbf{P}$ and $\mathbf{n}$, and the function f , find the scalar λ that minimizes $f(\mathbf{P} + \lambda\mathbf{n})$. Replace $\mathbf{P}$ by $\mathbf{P} + \lambda\mathbf{n}$. Replace $\mathbf{n}$ by $\lambda\mathbf{n}$. Done.

Since we will want to use `linmin` with methods whose choice of successive directions does not involve explicit computation of the function's gradient, `linmin` itself cannot use gradient information. Later, in §10.8, we will consider a method that does use gradient information. Accordingly, there we provide a routine `dlinmin` that makes use of this information to reduce the total computational burden.

The obvious way to implement `linmin` is to use the methods of one-dimensional minimization described in §10.2 – §10.4, but to rewrite the programs of those sections so that their bookkeeping is done on vector-valued points **P** (all lying along a given direction **n**) rather than scalar-valued abscissas *x*. That straightforward task produces long routines densely populated with “`for(k=0;k<n;k++)`” loops.

As an alternative, we can simply reuse the one-dimensional minimization routines by constructing a functor `F1dim`, which gives the value of your function, say `func`, along the line going through the point `p` in the direction `xi`. The function `linmin` calls our familiar one-dimensional routine `Brent` (§10.4) and instructs it to minimize `F1dim`. The routine `linmin` communicates with `F1dim` “over the head” of `Brent` through the constructor, our usual C++ idiom.

mins_ndim.h

```
template <class T>
struct Linemethod {
Base class for line-minimization algorithms. Provides the line-minimization routine linmin.
    VecDoub p;
    VecDoub xi;
    T &func;
    Int n;
    Linemethod(T &func) : func(func) {}
    Constructor argument is the user-supplied function or functor to be minimized.
    Doub linmin()
    Line-minimization routine. Given an n-dimensional point p[0..n-1] and an n-dimensional
    direction xi[0..n-1], moves and resets p to where the function or functor func(p) takes on
    a minimum along the direction xi from p, and replaces xi by the actual vector displacement
    that p was moved. Also returns the value of func at the returned location p. This is actually
    all accomplished by calling the routines bracket and minimize of Brent.
    {
        Doub ax,xx,xmin;
        n=p.size();
        F1dim<T> f1dim(p,xi,func);
        ax=0.0;
        xx=1.0;
        Brent brent;
        brent.bracket(ax,xx,f1dim);
        xmin=brent.minimize(f1dim);
        for (Int j=0;j<n;j++) {
            xi[j] *= xmin;
            p[j] += xi[j];
        }
        return brent.fmin;
    }
};
```

Initial guess for brackets.

Construct the vector results to return.

mins_ndim.h

```
template <class T>
struct F1dim {
Must accompany linmin in Linemethod.
    const VecDoub &p;
    const VecDoub &xi;
    Int n;
    T &func;
    VecDoub xt;
```

```

Fdim(VecDoub_I &pp, VecDoub_I &xii, T &funcc) : p(pp),
    xi(xii), n(pp.size()), func(funcc), xt(n) {}
Constructor takes as inputs an n-dimensional point p[0..n-1] and an n-dimensional di-
rection xi[0..n-1] from linmin, as well as the function or functor that takes a vector
argument.
Doub operator() (const Doub x)
Functor returning value of the given function along a one-dimensional line.
{
    for (Int j=0;j<n;j++)
        xt[j]=p[j]+x*xi[j];
    return func(xt);
}
};

```

10.7 Direction Set (Powell's) Methods in Multidimensions

With a routine for line minimization in hand, you might think of this simple method for general multidimensional minimization: Take the unit vectors $\mathbf{e}_0, \mathbf{e}_1, \dots, \mathbf{e}_{N-1}$ as a *set of directions*. Using `linmin`, move along the first direction to its minimum, then *from there* along the second direction to *its* minimum, and so on, cycling through the whole set of directions as many times as necessary, until the function stops decreasing.

This simple method is actually not too bad for many functions. Even more interesting is why it *is* bad, i.e., very inefficient, for some other functions. Consider a function of two dimensions whose contour map (level lines) happens to define a long, narrow valley at some angle to the coordinate basis vectors (see Figure 10.7.1). Then the only way “down the length of the valley” going along the basis vectors at each stage is by a series of many tiny steps. More generally, in N dimensions, if the function's second derivatives are much larger in magnitude in some directions than in others, then many cycles through all N basis vectors will be required in order to get anywhere. This condition is not all that unusual; according to Murphy's Law, you should count on it.

Obviously what we need is a better set of directions than the $\mathbf{e}_i$'s. All *direction set methods* consist of prescriptions for updating the set of directions as the method proceeds, attempting to come up with a set that either (i) includes some very good directions that will take us far along narrow valleys, or else (more subtly) (ii) includes some number of “noninterfering” directions with the special property that minimization along one is not “spoiled” by subsequent minimization along another, so that interminable cycling through the set of directions can be avoided.

10.7.1 Conjugate Directions

This concept of “noninterfering” directions, more conventionally called *conjugate directions*, is worth making mathematically explicit.

First, note that if we minimize a function along some direction $\mathbf{u}$, then the gradient of the function must be perpendicular to $\mathbf{u}$ at the line minimum; if not, then there would still be a nonzero directional derivative along $\mathbf{u}$.

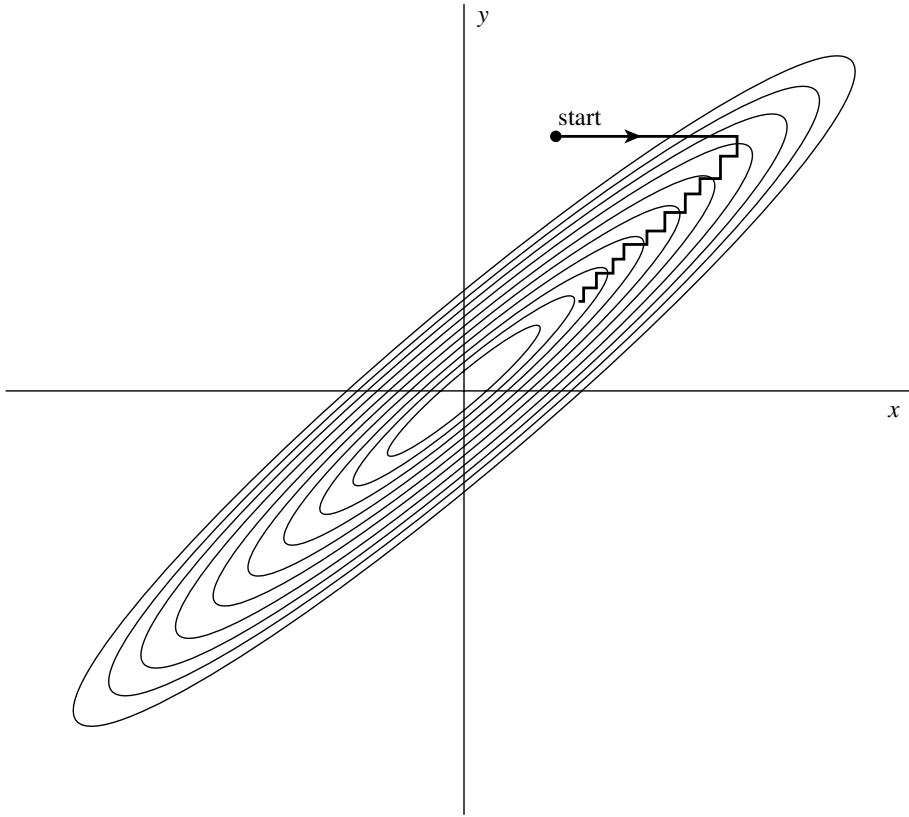


Figure 10.7.1. Successive minimizations along coordinate directions in a long, narrow “valley” (shown as contour lines). Unless the valley is optimally oriented, this method is extremely inefficient, taking many tiny steps to get to the minimum, crossing and re-crossing the principal axis.

Next take some particular point $\mathbf{P}$ as the origin of the coordinate system with coordinates $\mathbf{x}$. Then any function f can be approximated by its Taylor series

$$\begin{aligned} f(\mathbf{x}) &= f(\mathbf{P}) + \sum_i \frac{\partial f}{\partial x_i} x_i + \frac{1}{2} \sum_{i,j} \frac{\partial^2 f}{\partial x_i \partial x_j} x_i x_j + \cdots \\ &\approx c - \mathbf{b} \cdot \mathbf{x} + \frac{1}{2} \mathbf{x} \cdot \mathbf{A} \cdot \mathbf{x} \end{aligned} \quad (10.7.1)$$

where

$$c \equiv f(\mathbf{P}) \quad \mathbf{b} \equiv -\nabla f|_{\mathbf{P}} \quad [\mathbf{A}]_{ij} \equiv \left. \frac{\partial^2 f}{\partial x_i \partial x_j} \right|_{\mathbf{P}} \quad (10.7.2)$$

The matrix $\mathbf{A}$ whose components are the second partial derivative matrix of the function is called the *Hessian matrix* of the function at $\mathbf{P}$.

In the approximation of (10.7.1), the gradient of f is easily calculated as

$$\nabla f = \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \quad (10.7.3)$$

(This implies that the gradient will vanish — the function will be at an extremum — at a value of $\mathbf{x}$ obtained by solving $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. We will return to this idea in §10.9!)

How does the gradient ∇f change as we move along some direction? Evidently

$$\delta(\nabla f) = \mathbf{A} \cdot (\delta \mathbf{x}) \quad (10.7.4)$$

Suppose that we have moved along some direction $\mathbf{u}$ to a minimum and now propose to move along some new direction $\mathbf{v}$. The condition that motion along $\mathbf{v}$ not *spoil* our minimization along $\mathbf{u}$ is just that the gradient stay perpendicular to $\mathbf{u}$, i.e., that the change in the gradient be perpendicular to $\mathbf{u}$. By equation (10.7.4) this is just

$$0 = \mathbf{u} \cdot \delta(\nabla f) = \mathbf{u} \cdot \mathbf{A} \cdot \mathbf{v} \quad (10.7.5)$$

When (10.7.5) holds for two vectors $\mathbf{u}$ and $\mathbf{v}$, they are said to be *conjugate*. When the relation holds pairwise for all members of a set of vectors, they are said to be a conjugate set. If you do successive line minimizations of a function along a conjugate set of directions, then you don't need to redo any of those directions (unless, of course, you spoil things by minimizing along a direction that they are *not* conjugate to).

A triumph for a direction set method is to come up with a set of N linearly independent, mutually conjugate directions. Then, one pass of N line minimizations will put it exactly at the minimum of a quadratic form like (10.7.1). For functions f that are not exactly quadratic forms, it won't be exactly at the minimum, but repeated cycles of N line minimizations will in due course converge *quadratically* to the minimum.

10.7.2 Powell's Quadratically Convergent Method

Powell first discovered a direction set method that does produce N mutually conjugate directions. Here is how it goes: Initialize the set of directions $\mathbf{u}_i$ to the basis vectors,

$$\mathbf{u}_i = \mathbf{e}_i \quad i = 0, \dots, N-1 \quad (10.7.6)$$

Now repeat the following sequence of steps ("basic procedure") until your function stops decreasing:

- Save your starting position as $\mathbf{P}_0$.
- For $i = 0, \dots, N-1$, move $\mathbf{P}_i$ to the minimum along direction $\mathbf{u}_i$ and call this point $\mathbf{P}_{i+1}$.
- For $i = 0, \dots, N-2$, set $\mathbf{u}_i \leftarrow \mathbf{u}_{i+1}$.
- Set $\mathbf{u}_{N-1} \leftarrow \mathbf{P}_N - \mathbf{P}_0$.
- Move $\mathbf{P}_N$ to the minimum along direction $\mathbf{u}_{N-1}$ and call this point $\mathbf{P}_0$.

Powell, in 1964, showed that, for a quadratic form like (10.7.1), k iterations of the above basic procedure produce a set of directions $\mathbf{u}_i$ whose last k members are mutually conjugate. Therefore, N iterations of the basic procedure, amounting to $N(N+1)$ line minimizations in all, will exactly minimize a quadratic form. Brent [1] gives proofs of these statements in accessible form.

Unfortunately, there is a problem with Powell's quadratically convergent algorithm. The procedure of throwing away, at each stage, $\mathbf{u}_0$ in favor of $\mathbf{P}_N - \mathbf{P}_0$ tends to produce sets of directions that "fold up on each other" and become linearly dependent. Once this happens, the procedure finds the minimum of the function f only over a subspace of the full N -dimensional case; in other words, it gives the wrong answer. Therefore, the algorithm must not be used in the form given above.

There are a number of ways to fix up the problem of linear dependence in Powell's algorithm, among them:

1. You can reinitialize the set of directions $\mathbf{u}_i$ to the basis vectors $\mathbf{e}_i$ after every N or $N + 1$ iterations of the basic procedure. This produces a serviceable method, which we commend to you if quadratic convergence is important for your application (i.e., if your functions are close to quadratic forms and if you desire high accuracy).

2. Brent points out that the set of directions can equally well be reset to the columns of any orthogonal matrix. Rather than throw away the information on conjugate directions already built up, he resets the direction set to calculated principal directions of the matrix $\mathbf{A}$ (which he gives a procedure for determining). The calculation is essentially a singular value decomposition algorithm (see §2.6). Brent has a number of other cute tricks up his sleeve, and his modification of Powell's method is probably the best presently known. Consult [1] for a detailed description and listing of the program. Unfortunately it is rather too elaborate for us to include here.

3. You can give up the property of quadratic convergence in favor of a more heuristic scheme (due to Powell) that tries to find a few good directions along narrow valleys instead of N necessarily conjugate directions. This is the method that we now implement. (It is also the version of Powell's method given in Acton [2], from which parts of the following discussion are drawn.)

10.7.3 Discarding the Direction of Largest Decrease

The fox and the grapes: Now that we are going to give up the property of quadratic convergence, was it so important after all? That depends on the function that you are minimizing. Some applications produce functions with long, twisty valleys. Quadratic convergence is of no particular advantage to a program that must slalom down the length of a valley floor that twists one way and another (and another, and another, . . . — there are N dimensions!). Along the long direction, a quadratically convergent method is trying to extrapolate to the minimum of a parabola that just isn't (yet) there while the conjugacy of the $N - 1$ transverse directions keeps getting spoiled by the twists.

Sooner or later, however, we do arrive at an approximately ellipsoidal minimum (cf. equation 10.7.1 when $\mathbf{b}$, the gradient, is zero). Then, depending on how much accuracy we require, a method with quadratic convergence can save us several times N^2 extra line minimizations, since quadratic convergence *doubles* the number of significant figures at each iteration.

The basic idea of our now-modified Powell's method is still to take $\mathbf{P}_N - \mathbf{P}_0$ as a new direction; it is, after all, the average direction moved in after trying all N possible directions. For a valley whose long direction is twisting slowly, this direction is likely to give us a good run along the new long direction. The change is to discard the old direction along which the function f made its *largest decrease*. This seems paradoxical, since that direction was the *best* of the previous iteration. However, it is also likely to be a major component of the new direction that we are adding, so dropping it gives us the best chance of avoiding a buildup of linear dependence.

There are a couple of exceptions to this basic idea. Sometimes it is better *not* to add a new direction at all. Define

$$f_0 \equiv f(\mathbf{P}_0) \quad f_N \equiv f(\mathbf{P}_N) \quad f_E \equiv f(2\mathbf{P}_N - \mathbf{P}_0) \quad (10.7.7)$$